UNIVERSITY OF NAPLES FEDERICO II

POLYTECHNIC AND BASIC SCIENCES SCHOOL

DEPARTMENT OF INDUSTRIAL ENGINEERING

MASTER'S DEGREE IN
AUTONOMOUS VEHICLE ENGINEERING
MACHINE LEARNING AND BIG DATA

PROJECT REPORT

Mustafa Oner D18000070

ACADEMIC YEAR 2024-2025

---

1 Chosen Modelling Strategy & Motivation

The project stores the *IMDb Top-1000* film dataset in **MongoDB**, a document-oriented NoSQL database.
MongoDB was preferred because:

- **Flexible documents** – film records contain arrays (e.g. *genre*, *cast*) and optional fields (*meta_score*, *gross*). A rigid relational schema would require many join tables, whereas BSON documents can grow organically.
- **Schema-on-read** – missing values do not block loading; the Python ETL script inserts None for absent numbers and safely ignores gaps.
- **Rich aggregation pipeline** – complex calculations (group-by, unwind, sort, limit) run server-side, reducing data shipped to the client.
- **Developer productivity** – documents are JSON-like, matching the pandas→Python→MongoDB workflow in the code.

A single-collection design keeps each movie in one document, so typical analytical queries finish with one pipeline instead of multiple joins.

---

2 Database Structure (Document Schema)

A *movie* document looks like:

```
{
  "title": "Inception",
  "year": 2010,
  "certificate": "PG-13",
  "runtime_minutes": 148,
  "genre": ["Action", "Adventure", "Sci-Fi"],
  "rating": 8.8,
  "meta_score": 74,        // optional
  "director": "Christopher Nolan",
  "cast": [
    "Leonardo DiCaprio",
    "Joseph Gordon-Levitt",
    "Elliot Page",
    "Tom Hardy"
  ],
  "votes": 2000000,
  "gross": 829895144       // optional
}
```

Arrays support many-to-many relationships (e.g., genres, cast).
Optional numeric fields remain absent when the CSV cell is blank; the loader converts them to integers/floats where possible.

Indexes created

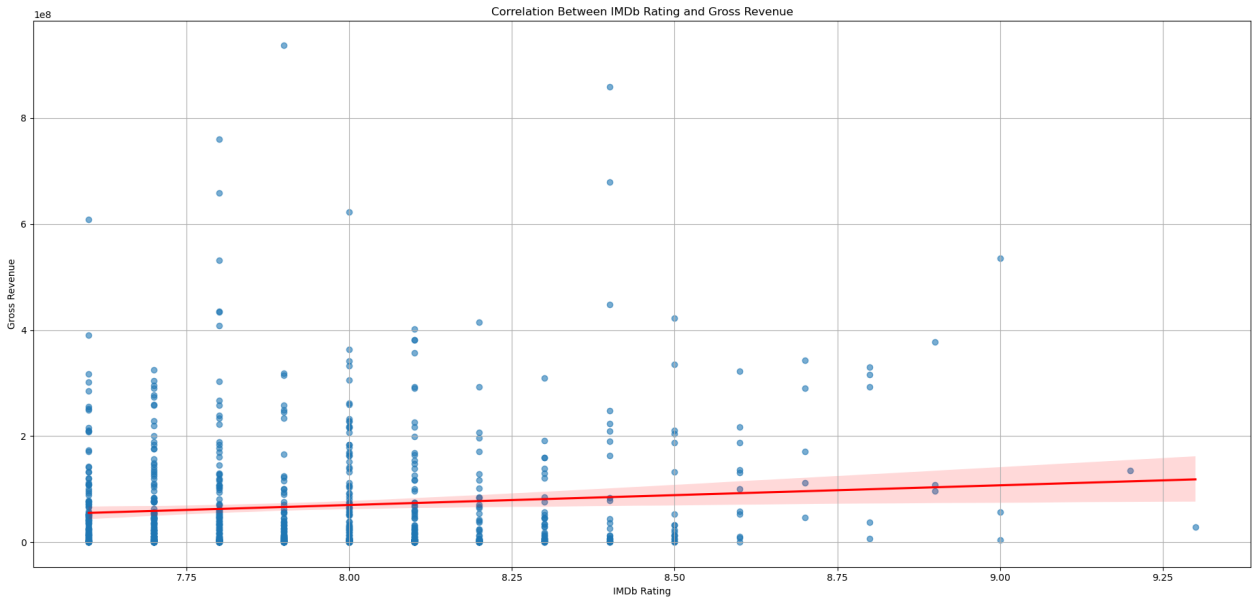
Field	Reason
genre (multikey)	speeds aggregation by genre
director	used in “top directors” query
cast (multikey)	used in actor appearances
compound rating + gross	accelerates correlation analysis

The left half of **Figure 1** summarises the schema visually.

---

3 Implemented Queries & Key Insights

#	Aggregation pipeline	Main finding
1	unwind genre → group → count	**Drama** dominates the list (> 700 titles), followed by Comedy and Crime.
2	unwind genre → group → avg(rating)	*War*, *Western* and *Film-Noir* show the highest average ratings (~ 8.0).
3	group by director → count	**Alfred Hitchcock** (14), **Steven Spielberg** (13) and **Hayao Miyazaki** (11) are the most frequent directors in the top 1000.
4	unwind cast → group → count	**Robert De Niro** appears 17 times, ahead of Tom Hanks and Al Pacino.
5	project rating, gross → corr()	Pearson correlation between *rating* and *gross* is low-positive (~ 0.18), so critical acclaim is not a strong guarantee of box-office success.



4 Challenges & Mitigation

Challenge	Solution
Missing or malformed data (e.g. empty *gross*, commas inside numbers)	ETL step parses numbers with replace() and assigns None when parsing fails.
Comma-separated lists in CSV (genres, cast)	Split strings into Python lists before insertion.
Long labels crowd plots	Matplotlib xticks(rotation=45) plus tight_layout() for readability.
Uneven attribute presence complicates indexes	Created indexes only on commonly populated fields; sparse index option if needed.
Securing credentials in code	In production, move URI to environment variables or secrets manager; the classroom script keeps them inline for simplicity.

Conclusion

Using a document-oriented approach with MongoDB let us load, query and visualise the IMDb Top-1000 dataset rapidly. The flexible schema captured rich movie metadata; aggregation pipelines produced actionable insights, and Python notebooks rendered clear graphics. The same pattern can scale to recommendation engines or streaming-media dashboards with minimal changes.

Project Link:
https://drive.google.com/file/d/14AVbdk189z85_XyrpfiGrgd37VPTWW35/view